CUDA Ray Tracing Project Report

Artur Quarra

April 14, 2026

1 Introduction

This project implements a Python ray tracer with both CPU and CUDA GPU execution paths. The renderer traces rays through a scene of spheres over a checkerboard plane, with recursive reflection and refraction, multiple light sources, and multiple predefined camera views. The final version supports batch rendering of several viewpoints and a full GPU path through Numba CUDA.

2 Objectives

The main goals were:

1. implement recursive ray tracing,
2. add plane intersection in addition to sphere intersection,
3. implement reflection,
4. implement refraction,
5. render the same scene from multiple viewpoints,
6. support both CPU and GPU execution.

3 Scene Description

The final scene consists of:

- three spheres with different material properties,
- one checkerboard plane,
- two light sources,
- five predefined camera viewpoints.

The spheres were chosen to demonstrate reflective and refractive effects clearly. The plane provides a visible reference surface for shadows, reflections, and perspective.

4 Mathematical Background

4.1 Ray Equation

A ray is written as

$$\mathbf{r}(t) = \mathbf{e} + t\mathbf{d}, \quad t \geq 0$$

where $\mathbf{e}$ is the origin and $\mathbf{d}$ is the direction.

4.2 Sphere Intersection

A sphere centered at $\mathbf{c}$ with radius r satisfies

$$\|\mathbf{p} - \mathbf{c}\|^2 = r^2$$

Substituting $\mathbf{p} = \mathbf{e} + t\mathbf{d}$ gives

$$\|\mathbf{e} + t\mathbf{d} - \mathbf{c}\|^2 = r^2$$

which expands to

$$at^2 + bt + c = 0$$

with

$$a = \mathbf{d} \cdot \mathbf{d}, \qquad b = 2(\mathbf{e} - \mathbf{c}) \cdot \mathbf{d}, \qquad c = (\mathbf{e} - \mathbf{c}) \cdot (\mathbf{e} - \mathbf{c}) - r^2$$

and discriminant

$$\Delta = b^2 - 4ac$$

If $\Delta < 0$, there is no hit. Otherwise the nearest positive root is used.

4.3 Plane Intersection

For a plane defined by point $\mathbf{p_0}$ and normal $\mathbf{n}$,

$$t = \frac{\mathbf{n} \cdot (\mathbf{p_0} - \mathbf{e})}{\mathbf{n} \cdot \mathbf{d}}$$

If the denominator is near zero, the ray is parallel to the plane.

4.4 Surface Normal

For a sphere, the normal is

$$\mathbf{n} = \frac{\mathbf{p} - \mathbf{c}}{\|\mathbf{p} - \mathbf{c}\|}$$

For a plane, the normal is constant.

4.5 Diffuse Lighting

Lambertian diffuse shading is

$$I_d = k_d \max(0, \mathbf{n} \cdot \mathbf{l})$$

4.6 Specular Lighting

Using the half vector

$$\mathbf{h} = \frac{\mathbf{l} + \mathbf{v}}{\|\mathbf{l} + \mathbf{v}\|}$$

the specular term is

$$I_s = k_s \max(0, \mathbf{n} \cdot \mathbf{h})^\alpha$$

4.7 Reflection

The reflection direction is

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n}$$

4.8 Refraction

Refraction follows Snell's law:

$$n_1 \sin\theta_1 = n_2 \sin\theta_2$$

The transmitted ray is computed from the incident direction, surface normal, and index of refraction. If total internal reflection occurs, the refracted ray is discarded.

4.9 Fresnel Approximation

Schlick's approximation was used:

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos\theta)^5$$

This blends reflection and refraction more realistically.

4.10 Recursive Ray Tracing

The recursive process is:

1. cast a primary ray,
2. find nearest intersection,
3. compute local shading,
4. spawn reflected rays if needed,
5. spawn refracted rays if needed,
6. accumulate color until the maximum depth is reached.

4.11 Camera Model

The camera basis is computed as

$$\mathbf{w} = \frac{\mathbf{eye} - \mathbf{lookAt}}{\|\mathbf{eye} - \mathbf{lookAt}\|}$$

$$\mathbf{u} = \frac{\mathbf{up} \times \mathbf{w}}{\|\mathbf{up} \times \mathbf{w}\|}$$

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}$$

4.12 Tone Mapping and Gamma

ACES-style tone mapping and gamma correction are applied:

$$f(x) = \frac{x(ax + b)}{x(cx + d) + e}$$

followed by

$$C_{\text{display}} = C_{\text{linear}}^{1/2.2}$$

5 Implementation

The project includes:

- a CPU fallback renderer in Python,
- a full CUDA path using Numba,
- a checkerboard plane,
- multiple view rendering,
- predefined render modes including preview, CPU, GPU, and 4K GPU.

The renderer supports multiple viewpoints automatically, which satisfies the scene-from-multiple-viewpoints requirement. The final code also includes predefined views such as front, right, left, wide, and close-up.

6 Results

The final outputs demonstrate:

- recursive reflection,
- recursive refraction,
- visible checkerboard reflections,

- multiple camera views of the same scene,
- GPU-accelerated rendering.

Observed properties of the final images:

- the pink sphere clearly reflects the checkerboard floor,
- the center sphere behaves like a mirror or glass object depending on material setup,
- the rear sphere provides depth and occlusion cues,
- the plane shows perspective and reflections clearly,
- the five views confirm that the camera system is working correctly.

7 Example Figures

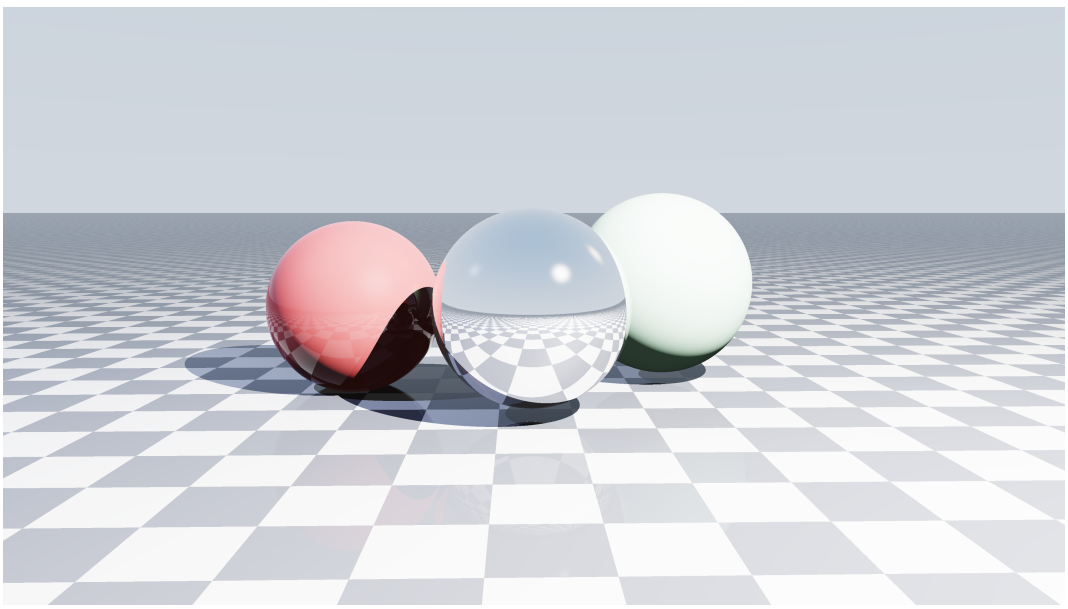

Figure 1: Front view of the scene.

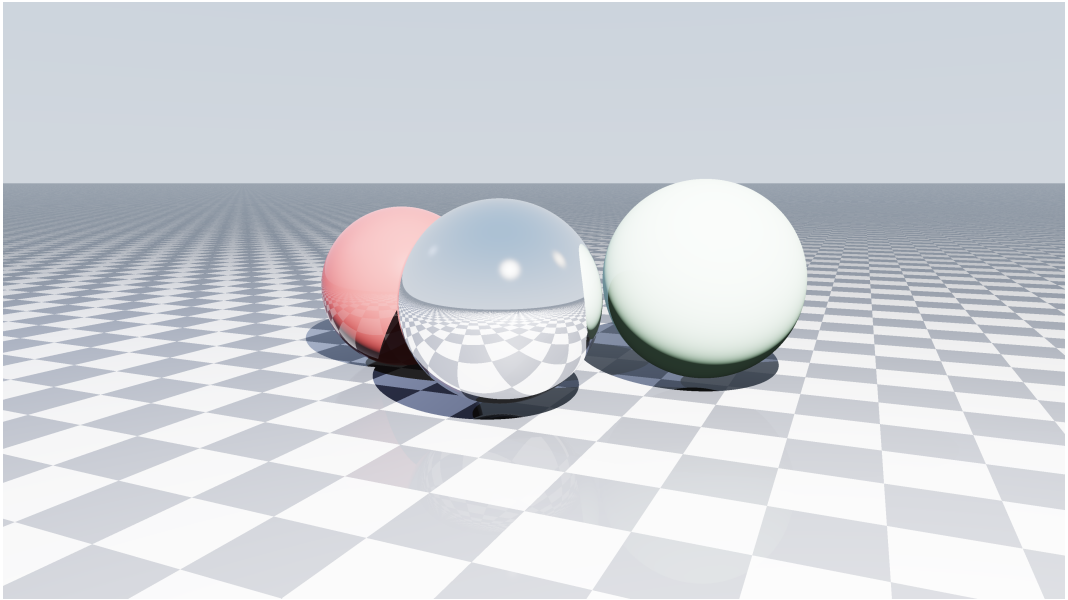

Figure 2: Right-side view of the scene.

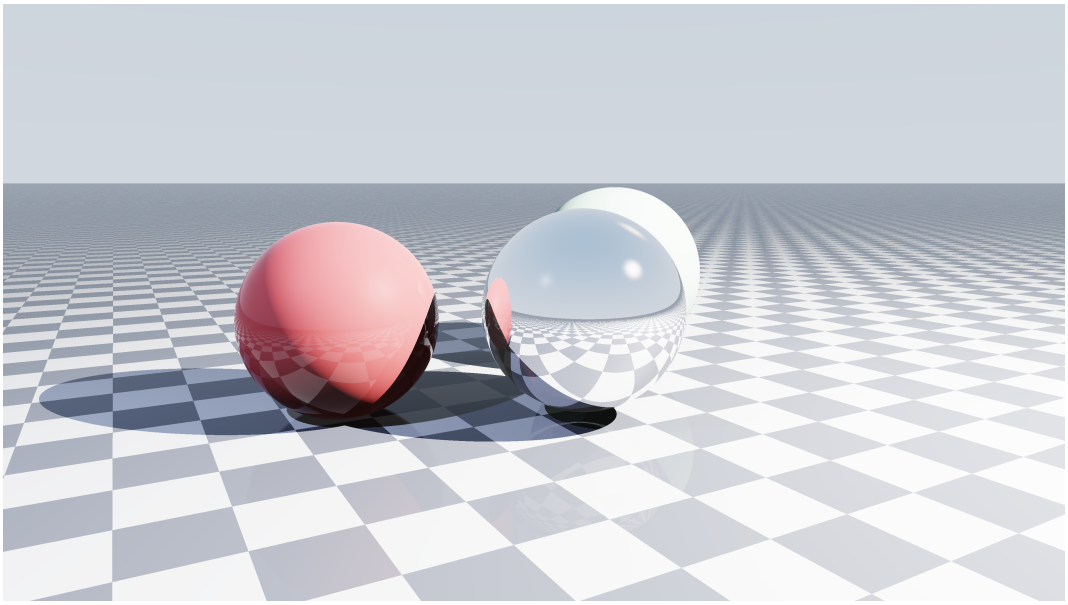

Figure 3: Left-side view of the scene.

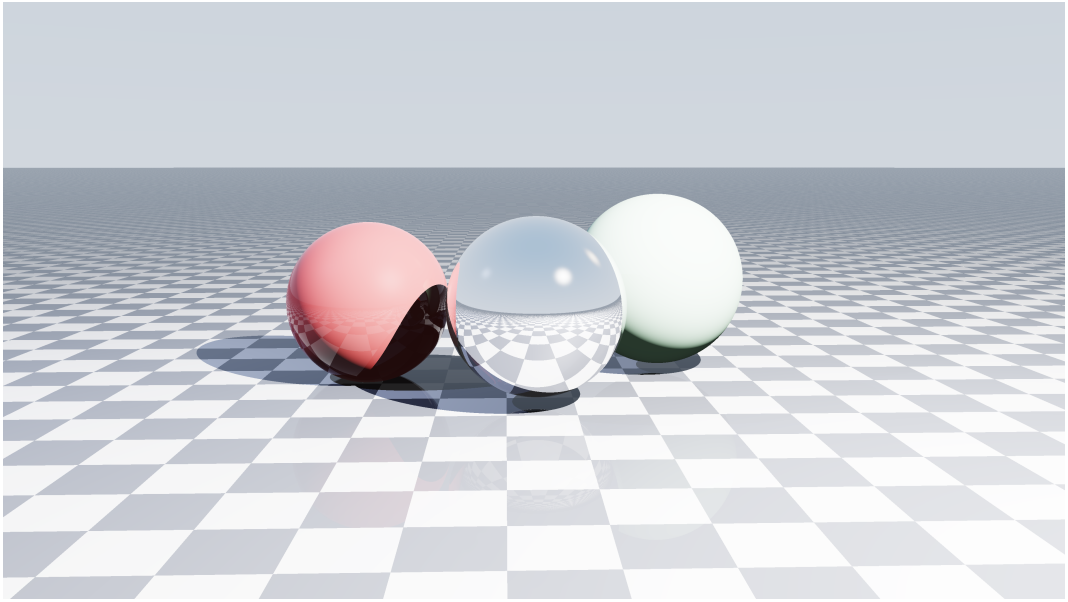

Figure 4: Wide-angle view of the scene.

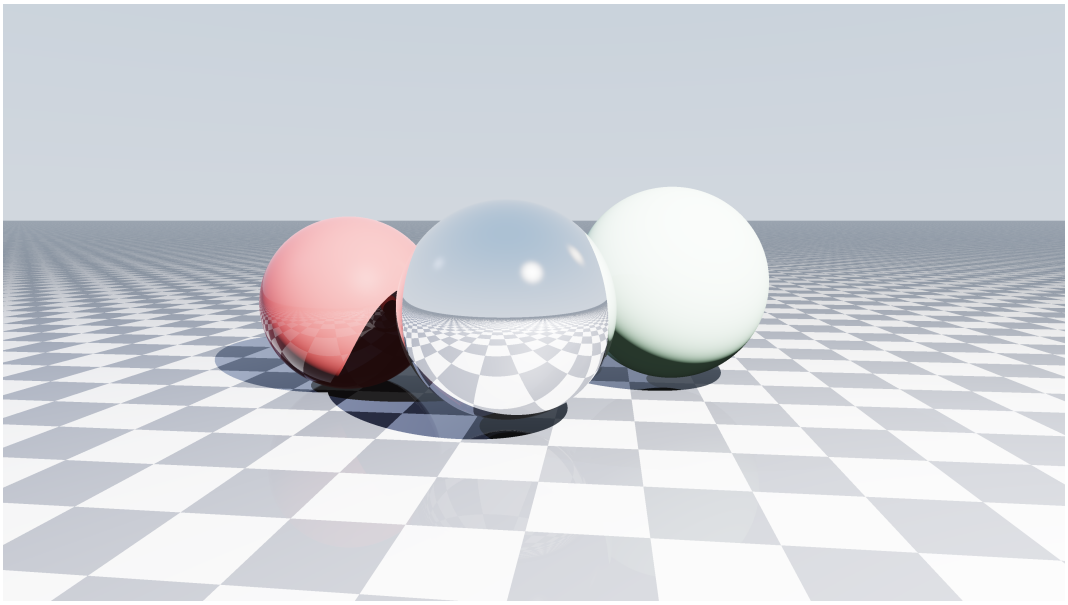

Figure 5: Close-up view of the scene.

8 Requirement Mapping

Requirement	Status
Basic recursive ray tracing	Implemented
Reflection	Implemented
Refraction	Implemented
Rendering from multiple viewpoints	Implemented
Plane intersection	Implemented
Phong/specular-style local shading	Implemented
Glossy reflection	Partially approximated
Soft shadows	Simplified / limited in current version

9 Challenges

Major challenges included:

- producing brighter and more realistic output,
- reducing CPU render time,
- configuring multiprocessing behavior,
- setting up CUDA and Numba correctly,
- resolving missing CUDA libraries such as `libcudart.so`, `libnvvm.so`, and `libdevice`,
- tuning the scene and materials so the final GPU outputs looked correct.

10 Limitations

Current limitations include:

- the scene is restricted mainly to spheres and a plane,
- glossy reflection is only partially approximated,
- soft shadows are simplified,
- a separate interactive 3D UI is not implemented in the provided renderer code.

11 Future Work

Future improvements could include:

- triangle mesh support,
- BVH acceleration,

- true soft-shadow sampling,
- a full glossy reflection model,
- texture mapping,
- an interactive 3D UI for scene editing and object movement.

12 Conclusion

This project successfully demonstrates a recursive ray tracer implemented in Python and accelerated with CUDA through Numba. The final system supports sphere and plane intersection, reflections, refractions, multiple viewpoints, and GPU rendering. The final outputs clearly show the scene from several camera angles and provide a strong demonstration of the mathematical and practical foundations of ray tracing.